

DataHarvest

Rapport technique

Karam DHIF & Ryma DINARI

Master Dev, Data & IA — 4e annee — IPSSI NICE

Projet final Web Scraping — DataHarvest

31 juillet 2026

1. Introduction et perimetre

DataHarvest répond a un problème récurrent du scraping artisanal : chaque nouveau site nécessite, en général, un script dédié qui mélange requête HTTP, parsing et sauvegarde dans un seul fichier. Ce couplage rend le code difficile a tester, a faire évoluer et a réutiliser d'un projet a l'autre.

DataHarvest propose l'inverse : un cœur générique (fetch, parse, valide, stocke) pilote entièrement par un fichier de configuration déclaratif. Ajouter un nouveau site revient a écrire un fichier YAML de selecteurs CSS, jamais une ligne de code Python.

Périmètre volontairement exclu

DataHarvest cible exclusivement les sites HTML statiques accessibles via une simple requête GET. Le rendu JavaScript côté client, l'authentification par formulaire, le défilement infini et l'exécution asynchrone sont hors périmètre : ils demanderaient soit un moteur de rendu (Selenium/Playwright), soit une architecture événementielle (asyncio/Twisted) qui aurait complexifié chaque composant pour un gain hors sujet dans le temps imparti (5 heures).

Sites choisis pour les tests

Cinq sites ont été retenus, couvrant quatre niveaux de difficulté différents sur les quatre proposés par le sujet (le minimum exige n'étant que de deux) : books.toscrape.com et quotes.toscrape.com (niveau 1, structure prévisible), fr.wikipedia.org sur des pages de données démographiques (niveau 2, tableaux structurés), blogdumoderateur.com (niveau 3, media d'actualité avec pagination par chemin d'URL) et github.com/trending (niveau 4, selecteurs plus complexes, page unique sans pagination classique).

Ce choix permet de vérifier que le même Framework s'adapte a la fois a des schémas de pagination différents (chemin d'URL pour blogdumoderateur, page unique pour github/trending) et aux trois backends de stockage (sqlite, json, csv) sans aucune modification du code source — seul le fichier de configuration change d'un site a l'autre :

Site	Niveau	Backend Store	Fichier de sortie
books.toscrape.com	1	sqlite	output/books_toscrape.db
quotes.toscrape.com	1	json	output/quotes_toscrape.json
fr.wikipedia.org (donnees de population)	2	json	output/wikipedia_population.json
blogdumoderateur.com	3	sqlite	output/blogdumoderateur.db
github.com/trending	4	csv (via export_to)	output/github_trending.csv

Le backend csv a été vérifié via export_to() a partir d'un backend sqlite, démontrant l'interopérabilité des trois formats de sortie sans code supplémentaire.

Les sites sélectionnés couvrent plusieurs niveaux de difficulté afin d'évaluer l'adaptabilité de DataHarvest. Les sites de **niveau 1** (*books.toscrape.com*, *quotes.toscrape.com*), le site structuré **fr.wikipedia.org** (niveau 2) et **github.com/trending** (niveau 4) n'ont pas présenté de difficulté majeure : les structures HTML étaient stables et les sélecteurs CSS définis dans les fichiers YAML ont fonctionné avec peu d'ajustements.

Le site **blogdumoderateur.com** (niveau 3 – médias et actualités) a demandé davantage de travail. La pagination `/page/{n}/` a nécessité une amélioration de la détection de fin de parcours, car le site retournait une réponse HTTP 200 même lorsque la page était vide. La condition d'arrêt a donc été complétée par une vérification du nombre d'articles extraits. Certains sélecteurs, notamment la catégorie (`.cat-links a`) et la date (`time[datetime]`), ont également dû être vérifiés avec DevTools afin de gérer les variations de structure HTML.

Ces tests montrent que DataHarvest peut s'adapter à des sites de niveaux différents tout en conservant une approche générique basée sur la configuration YAML.

2. Architecture et choix de conception

2.1 Pourquoi une architecture en composants decouplés ?

Un script monolithique fonctionne pour un site, mais chaque site suivant oblige à copier-coller et adapter le même code : la logique de retry, le parsing, la deduplication se retrouvent dupliqués et divergent silencieusement au fil du temps. En decoupant DataHarvest en cinq composants indépendants (Config, Fetcher, Pipeline, Validator, Store), chaque brique a une responsabilité unique et un contrat d'interface stable :

- Testabilité : chaque composant se teste isolément avec des entrées/sorties simples (un dict, une liste, une chaîne HTML), sans avoir à monter tout le pipeline ni à faire de vraies requêtes réseau (voir `tests/test_fetcher.py`, entièrement mocké).
- Extensibilité : ajouter un backend de stockage (ex : PostgreSQL) ne touche que `store.py` ; ajouter un middleware ne touche que `middleware.py`. Aucun autre fichier n'a besoin d'être modifié.
- Lisibilité : un fichier de moins de 150 lignes avec une seule responsabilité se lit et se relit plus vite qu'un script de 800 lignes qui fait tout.

2.2 Pourquoi BasePipeline est une ABC et non une classe à méthodes vides ?

Une classe normale avec des méthodes qui ne font rien laisse l'erreur se produire très loin de sa cause : si une sous-classe oublie d'implémenter `next_page_url()`, le bug n'apparaît qu'à l'exécution, quand l'Orchestrator appelle une méthode qui retourne toujours `None` silencieusement, sans jamais lever d'erreur explicite. En héritant d'`abc.ABC` et en marquant `process()` et `next_page_url()` avec `@abstractmethod`, Python refuse purement et simplement d'instancier une sous-classe incomplète : l'erreur remonte dès la construction de l'objet, avec un message explicite.

C'est exactement ce qui s'est produit pendant le développement de `GenericPipeline`, qui a dû recevoir une implémentation par défaut de `next_page_url()` (retournant toujours `None`) pour rester instanciable, `PaginationPipeline` la surchargeant ensuite avec la vraie logique. La garantie apportée à l'utilisateur du framework est donc contractuelle plutôt que documentaire : le contrat d'interface est vérifié par l'interpréteur, pas seulement suggéré par un commentaire.

2.3 Pourquoi le pattern middleware pour le Fetcher ?

Le même mécanisme est utilisé par Django et Flask pour intercepter requêtes/réponses sans modifier le cœur du Framework. Appliqué au Fetcher, il permet d'ajouter logging, retry ou rate-limiting comme des briques indépendantes, activables ou non selon la config, sans jamais toucher à `fetch()`.

Sans ce pattern, gérer le retry aurait obligé à coder le backoff exponentiel directement dans `fetch()`, en mélangeant logique de téléchargement et politique de résilience dans la même méthode, rendant impossible de tester la logique de retry indépendamment d'une vraie requête HTTP. Avec `RetryMiddleware` isolé, les tests vérifient le calcul du délai exponentiel et la décision de retry en pur Python, sans mock réseau.

2.4 Pourquoi la configuration via YAML plutôt que des arguments CLI ?

Un fichier de configuration YAML capture un état complet et versionnable : les selecteurs CSS, les règles de pagination, les paramètres de fetch, tout se lit et se modifie sans relancer une longue commande. C'est aussi la seule option raisonnable des qu'un site nécessite plus de trois ou quatre champs de selecteurs — une commande CLI avec vingt options deviendrait illisible. À l'inverse, les arguments CLI restent préférables pour des paramètres ponctuels qui ne méritent pas d'être persistés (ex : `--dry-run`, le chemin d'export). DataHarvest applique cette distinction : la configuration structurelle du scraping vit en YAML, les options d'exécution ponctuelles vivent en CLI.

2.5 Pourquoi l'injection de dépendances plutôt que des imports directs ?

Si `Fetcher` importait directement `RetryMiddleware` et l'instanciait lui-même en interne, il serait impossible de tester `fetch()` sans déclencher le vrai comportement de retry (attentes réelles, vraies exceptions requests). En recevant sa liste de middlewares au constructeur, le `Fetcher` ne connaît que l'interface `BaseMiddleware`, jamais une implémentation concrète. Concrètement, `tests/test_fetcher.py` instancie un `Fetcher` avec une session mockée et un `RetryMiddleware` à `base_delay=0.0` pour que le test s'exécute en quelques millisecondes au lieu d'attendre réellement le backoff exponentiel.

Le même raisonnement s'applique à l'`Orchestrator` lui-même : il reçoit un objet `Config` déjà chargé et valide, plutôt que de charger un chemin de fichier en interne. Un test unitaire peut ainsi construire une `Config` à partir d'un fichier temporaire et l'injecter dans un `Orchestrator` dont on mocke ensuite uniquement la méthode `fetch()` du `Fetcher`, isolant complètement le test de toute dépendance réseau tout en exerçant l'intégralité de la logique métier (pagination, validation, déduplication, comptage du rapport de session).

2.6 Flux de données et gestion des erreurs

L'`Orchestrator` centralise délibérément toute la gestion des erreurs de haut niveau : si le `Fetcher` lève une `FetchError` (retries épuisés), la boucle de pagination s'arrête proprement et retourne un rapport partiel plutôt que de laisser l'exception remonter et interrompre brutalement le processus. À l'inverse, les erreurs de plus bas niveau (un selecteur CSS qui ne matche rien, un champ manquant) ne remontent jamais d'exception : `GenericPipeline` retourne une chaîne vide pour un champ absent, et `Validator` se contente de rejeter l'item concerné en le loggant.

2.7 Convention de nommage des champs dans la Pipeline

`GenericPipeline` applique une convention plutôt qu'une configuration explicite pour déterminer quelle partie d'un tag HTML extraire : le champ nommé `'url'` récupère l'attribut `href` d'un lien, un tag possédant un attribut `datetime` le retourne tel quel, et tout le reste récupère le texte visible. Cette approche minimise la verbosité des fichiers de configuration, au prix d'une règle implicite qu'il faut documenter clairement — c'est l'objet de la section 5.2 de ce rapport.

2.8 Compromis architecturaux assumés

- Couplage volontaire dans l'Orchestrator : contrairement aux quatre autres composants, l'Orchestrator connaît et instancie directement Fetcher, Pipeline, Validator et Store. Ce couplage centralisé est assumé : concentrer l'assemblage en un seul point d'entrée évite de disperser cette logique à travers plusieurs fichiers.
- Pas de cache de requêtes : chaque exécution retelecharge intégralement les pages, y compris celles déjà visitées. Un cache réduirait la charge sur le site cible mais complexifierait Fetcher — juge hors scope pour la durée du projet.
- Validation binaire plutôt que graduée : Validator classe chaque item en valide ou rejeté, sans notion de score de confiance intermédiaire, ce qui simplifie l'implémentation mais perd une information potentiellement utile pour affiner les sélecteurs CSS a posteriori.

3. Comparaison avec Scrapy

3.1 Fonctionnalités de Scrapy absentes de DataHarvest

Fonctionnalité Scrapy	Raison de l'absence dans DataHarvest
Moteur asynchrone (Twisted/asyncio)	Hors scope pour 5h de développement : réécrire Fetcher et Orchestrator en asynchrone aurait multiplié la complexité (gestion des coroutines, synchronisation du Store) pour un gain de performance non prioritaire sur un projet pédagogique.
Middleware de gestion des cookies/sessions avancée	requests.Session() couvre les besoins de base (persistance des cookies) ; la gestion fine (rotation de sessions, cookies par domaine) est hors scope.
Item pipelines chainables avec priorités	DataHarvest a une seule Pipeline par config ; le chaînage de plusieurs étapes de traitement n'a pas été implémenté faute de temps, bien que l'architecture le permettrait facilement (liste de pipelines au lieu d'une seule).
Middleware de rotation de proxies/User-Agent	Complexité d'implémentation (pool de proxies, détection de bannissement) hors scope pour la durée du projet.
Scrapy Shell (console interactive de test des sélecteurs)	Outil de productivité important mais indépendant du cœur fonctionnel : priorité donnée aux composants métier plutôt qu'à l'outillage annexe.

3.2 Quand préférer DataHarvest à Scrapy ?

Scénario 1 — prototypage rapide sur un site unique : un data analyst qui doit extraire ponctuellement les prix d'un catalogue pour une étude, sans investir dans l'apprentissage du framework Scrapy (spiders, settings.py, items.py, pipelines.py), préfère écrire un fichier YAML de vingt lignes et lancer une commande.

Scénario 2 — intégration dans un pipeline data existant déjà synchrone : une équipe qui alimente déjà un ETL en Python synchrone (pandas, SQLAlchemy) intègre plus simplement DataHarvest, dont l'API Orchestrator.run() -> dict s'appelle comme une fonction Python ordinaire, sans avoir à faire cohabiter une boucle d'événements Twisted avec le reste du pipeline.

3.3 Impact du synchrone vs asynchrone : calcul théorique

Avec DOWNLOAD_DELAY=1s, scraper 100 pages en synchrone (DataHarvest, une requête à la fois) prend au minimum $100 \times 1s = 100$ secondes, plus le temps de réponse réseau de chaque page (à titre d'exemple 0,3s en moyenne), soit environ 130 secondes. Scrapy, avec c CONCURRENCE_LIMIT=16 (valeur par défaut) et le même délai appliqué par domaine plutôt que

par requete individuelle, peut traiter jusqu'a 16 pages en parallele : le temps total se rapproche de $100 / 16 \times (1s + 0,3s) \approx 8,1$ secondes, soit un gain d'environ 16x sur ce scenario.

Ce calcul reste theorique : en pratique, la concurrence de Scrapy est plafonnee par `CONCURRENT_REQUESTS_PER_DOMAIN` (8 par default), ce qui evite de saturer un seul serveur cible. Sur un scraping mono-domaine comme les cas d'usage de ce projet, le gain reel se rapproche donc davantage d'un facteur 8 que 16. Le gain de Scrapy ne se paie pas seulement en complexite de code : il augmente aussi la charge instantanee imposee au serveur cible, ce qui rejoint la question du cadre ethique traitee en section 4.

3.4 Courbe d'apprentissage et cout d'entree

Scrapy impose ses propres conventions (structure de projet generee par scrapy startproject, classes Spider avec des methodes de callback, settings.py centralisant des dizaines d'options) : un developpeur doit assimiler ce vocabulaire avant de pouvoir ecrire son premier spider fonctionnel. DataHarvest, plus modeste, repose sur des concepts deja familiers a tout developpeur Python et un fichier de configuration dont la structure se comprend en quelques minutes de lecture. DataHarvest est delibere pour couvrir 80% des besoins de scraping simple avec 20% de la complexite d'apprentissage de Scrapy, plutot que de viser l'exhaustivite fonctionnelle.

4. Cadre legal et ethique

4.1 Analyse du robots.txt des sites scrapes

books.toscrape.com et quotes.toscrape.com : ces deux sites, concus specifiquement pour l'entrainement au scraping, ne publient aucun fichier robots.txt (requete /robots.txt renvoyant une erreur 404). En l'absence de fichier, la convention du protocole d'exclusion des robots considere que l'integralite du site est accessible a l'exploration ; aucune regle n'a donc ete violee.

github.com : le robots.txt de github.com interdit explicitement plusieurs chemins parametres et sous-pages de depot (/*/pulse, /*/tree/, /*/commits/, /*/graphs, etc.), ainsi que certains parametres de requete (source=, ref=, author=...). La page /trending elle-meme, ciblee par la configuration configs/github_trending.yaml, ne figure pas parmi ces chemins interdits : le scraping realise reste donc conforme au fichier.

fr.wikipedia.org : le robots.txt de Wikipedia interdit l'exploration des espaces techniques et administratifs (/w/, /api/, /wiki/Special: et son equivalent francophone /wiki/Specialé:, ainsi que diverses pages de discussion administrative comme Wikipedia:Administrateur/Probleme/). Les pages d'articles standards sous /wiki/ consultees pour les donnees de population ne sont couvertes par aucune de ces interdictions.

blogdumoderateur.com: le fichier `robots.txt` du site a été consulté avant le scraping. Les règles d'exclusion concernent principalement des chemins techniques ou internes du site. Les pages d'articles utilisées par DataHarvest, accessibles via la pagination `/page/{n}/`, ne correspondent pas aux chemins interdits indiqués dans le fichier. Le scraping réalisé respecte donc les consignes du site. De plus, DataHarvest limite son impact grâce à un délai configurable entre les requêtes (`delay: 1.5`), un User-Agent identifiable et un nombre de tentatives limité en cas d'erreur.

4.2 Base legale RGPD (article 6)

L'article 6 du RGPD exige une base legale pour tout traitement de donnees a caractere personnel. Pour les cinq sites retenus (catalogue de livres, citations d'auteurs, depots logiciels tendance, donnees demographiques Wikipedia, articles de blog), les donnees collectees sont majoritairement

des données publiques à caractère non personnel (titres, prix, notes, catégories, statistiques, tags). Lorsque des données pourraient toucher des personnes physiques identifiables — le nom de l'auteur d'une citation sur quotes.toscrape.com, ou le nom d'utilisateur du propriétaire d'un dépôt sur github.com/trending — la base légale mobilisable est l'intérêt légitime (article 6.1.f) : la collecte poursuit un objectif pédagogique et documente, proportionne, sans profilage ni réutilisation commerciale, et les données en question sont déjà rendues publiques par la personne elle-même ou par la plateforme qui les publie.

4.3 Les données collectées sont-elles des données personnelles ?

Au sens de l'article 4.1 du RGPD, une donnée personnelle est toute information se rapportant à une personne physique identifiée ou identifiable. Les prix, notes, catégories et tags collectés sur books.toscrape.com, les statistiques démographiques de Wikipédia ainsi que les métadonnées de langage et d'étoiles sur github.com/trending ne se rapportent à aucune personne. Le nom d'auteur affiché sur une citation (quotes.toscrape.com) ou le nom d'utilisateur/organisation propriétaire d'un dépôt (github.com/trending) peuvent en revanche constituer une donnée à caractère personnel si la personne est identifiable à partir de ce seul nom, même lorsqu'il s'agit d'un pseudonyme professionnel déjà public. DataHarvest ne collecte cependant aucune donnée de contact, de profil ou de comportement associée à ces noms, ce qui limite fortement le risque pour les personnes concernées.

4.4 Mécanismes techniques de 'bon citoyen du web'

- Throttling configurable : `config.fetcher.delay` applique entre chaque requête (`fetch_all()`), et `RateLimitMiddleware` garantit un délai minimum par domaine même en cas d'appels concurrents.
- User-Agent identifiable et contactable : chaque config utilise "DataHarvest/1.0 (+contact@ipssi.fr)", jamais le User-Agent par défaut de la librairie `requests`, permettant à l'administrateur du site cible d'identifier et de contacter l'origine du trafic.
- Contrainte `UNIQUE(url)` en base : le backend `sqlite` refuse les doublons par URL (`INSERT OR IGNORE`), évitant de re-solliciter inutilement le site pour des données déjà collectées lors d'exécutions successives — vérifie concrètement sur `books_toscrape.db` et `blogdumoderateur.db`.
- Retries bornes avec backoff exponentiel : `RetryMiddleware` limite le nombre de tentatives (`max_retries` configurable) et espace les nouvelles requêtes exponentiellement plutôt que de marteler un serveur qui répond déjà en erreur (429/5xx).

5. Difficultés et retrospective

5.1 Difficulté technique principale

La difficulté la plus structurante a été la conception de `GenericPipeline._extract_value` : extraire une valeur depuis un tag BeautifulSoup selon une convention générique applicable à tous les sites (texte visible par défaut, attribut `href` pour le champ url, attribut `datetime` lorsqu'il est présent) sans développer de logique spécifique par site.

Une première version récupérait systématiquement l'attribut `href` lorsqu'un élément `<a>` était détecté, ce qui provoquait des erreurs lorsqu'un même sélecteur CSS était utilisé pour plusieurs champs, notamment titre (qui nécessite le texte visible) et url (qui nécessite le lien). La correction a

consisté à baser la logique d'extraction sur le nom du champ demandé plutôt que uniquement sur le type de balise HTML.

Lors de l'application de DataHarvest aux cinq sites sélectionnés, plusieurs ajustements ont également été nécessaires :

books.toscrape.com et quotes.toscrape.com : ces sites d'entraînement ont été les plus simples à intégrer. Les adaptations ont principalement concerné la définition des sélecteurs CSS et la gestion de la pagination.

fr.wikipedia.org : la difficulté principale a été la variation de structure des tableaux et des infobox selon les pages consultées, nécessitant une sélection précise des éléments HTML à extraire.

github.com/trending : l'inspection avec DevTools a été nécessaire afin d'identifier correctement les blocs contenant le nom du dépôt, la description, le langage utilisé et le nombre d'étoiles, la structure étant plus complexe que celle des sites d'entraînement.

blogdumoderateur.com : la principale difficulté concernait la pagination. Le site retournait un code HTTP 200 même lorsque la page ne contenait plus d'articles, ce qui a nécessité une amélioration de la condition d'arrêt dans `next_page_url()`. Certains champs, comme la catégorie et la date, présentaient également des variations selon les articles.

Ces différentes expériences ont confirmé l'intérêt d'une architecture configurable : les adaptations ont principalement été réalisées dans les fichiers YAML, sans modification du cœur du framework.

5.2 Ce qui serait fait différemment

L'extraction de valeur par convention de nommage de champ (section 5.1) reste fragile : elle suppose implicitement qu'un champ nommé 'url' pointe toujours vers un `<a href>`. Une version future gagnerait à expliciter le type d'extraction dans la config elle-même (ex : `url: {selector: 'h2.title a', attr: 'href'}`) plutôt qu'une simple chaîne), rendant le comportement visible dans le YAML plutôt qu'implicite dans le code Python.

5.3 Fonctionnalité non implémentée faute de temps

Le `RateLimitMiddleware` (extension bonus) est implémenté et testé au niveau unitaire, mais l'Orchestrator ne l'active pas par défaut dans sa chaîne de middlewares — seuls `LoggingMiddleware` et `RetryMiddleware` y sont instanciés en dur. Une implémentation complète rendrait la liste de middlewares elle-même configurable depuis le YAML, permettant d'activer ou désactiver chaque brique sans modifier `orchestrator.py`.

5.4 Répartition des tâches

Le binôme **Karam DHIFI et Ryma DINARI** a travaillé en collaboration sur l'ensemble du projet DataHarvest. Contrairement à une séparation stricte des composants, les différentes étapes du développement ont été réalisées conjointement : conception de l'architecture, implémentation des composants, configuration des sites, tests et rédaction de la documentation.

La répartition du travail s'est principalement organisée autour d'une validation croisée : chaque membre a participé au développement et à la vérification des différentes briques du framework (Config, Fetcher, Middleware, Pipeline, Validator, Store, Orchestrator et CLI). Les choix techniques, les corrections de bugs et les ajustements des fichiers YAML ont été discutés et réalisés en commun.

Cette organisation a permis d'assurer une compréhension globale du projet par les deux membres du binôme, plutôt qu'une connaissance limitée à une seule partie du code. La contribution de chacun est visible dans l'historique Git du dépôt à travers les différents commits réalisés par les deux auteurs.

6. Perspectives

6.1 Une sixieme brique : Notifier

Un composant Notifier completerait naturellement le flux de l'Orchestrator en envoyant une alerte (email, webhook Slack/Discord) a la fin de chaque session de scraping, notamment en cas d'echec ou de chute anormale du nombre d'items collectes par rapport aux executions precedentes — un signal frequent de changement de structure HTML sur le site cible.

```
class BaseNotifier(ABC):
    @abstractmethod
    def notify(self, report: dict) -> None:
        """Envoie le rapport de session vers un canal externe."""
```

Comme les autres composants, Notifier s'injecterait dans l'Orchestrator au constructeur (Orchestrator(config, notifiers=[EmailNotifier(...)]), sans creer de dependance directe entre orchestrator.py et une implementation concrete de notification.

6.2 Compatibilite avec les sites JavaScript sans Selenium

Selenium pilote un navigateur complet, ce qui le rend lourd. Deux alternatives plus legeres existent : Playwright, qui offre une API similaire mais avec un mode headless plus performant ; et le pre-rendu via un service comme Splash, expose comme une simple API HTTP interrogeable par le Fetcher existant sans le modifier. DataHarvest pourrait ajouter un PlaywrightFetcher implementant la meme interface que Fetcher, permettant a l'Orchestrator de choisir dynamiquement le fetcher a utiliser selon un champ `fetcher.engine: requests|playwright` dans la config, sans modifier Pipeline, Validator ni Store.

6.3 Distribution sur PyPI

Publier DataHarvest sur PyPI necessiterait : un fichier `pyproject.toml` declarant les metadonnees du paquet (nom, version, dependances, point d'entree CLI via `[project.scripts]`) ; le retrait de tout chemin absolu ou relatif code en dur ; une couverture de tests etendue aux cas limites non couverts par le present rapport (encodages non-UTF-8, HTML malforme) ; une politique de versionnement semantique explicite ; et un packaging verifie via `python -m build` puis `twine upload` vers PyPI, apres tests sur TestPyPI.

Bibliographie / Sources

1. Documentation officielle Scrapy — <https://docs.scrapy.org/>
2. Documentation officielle Requests — <https://requests.readthedocs.io/>
3. Documentation officielle BeautifulSoup 4 — <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>
4. Reglement General sur la Protection des Donnees (RGPD), articles 4 et 6 — <https://eur-lex.europa.eu/eli/reg/2016/679/oj>

5. Documentation Playwright pour Python — <https://playwright.dev/python/>
6. PEP 517 / packaging.python.org — guide de publication d'un paquet sur PyPI
7. Manuel MediaWiki, Robots.txt — <https://www.mediawiki.org/wiki/Manual:Robots.txt> et fr.wikipedia.org/robots.txt
8. GitHub, fichier robots.txt public — <https://github.com/robots.txt>